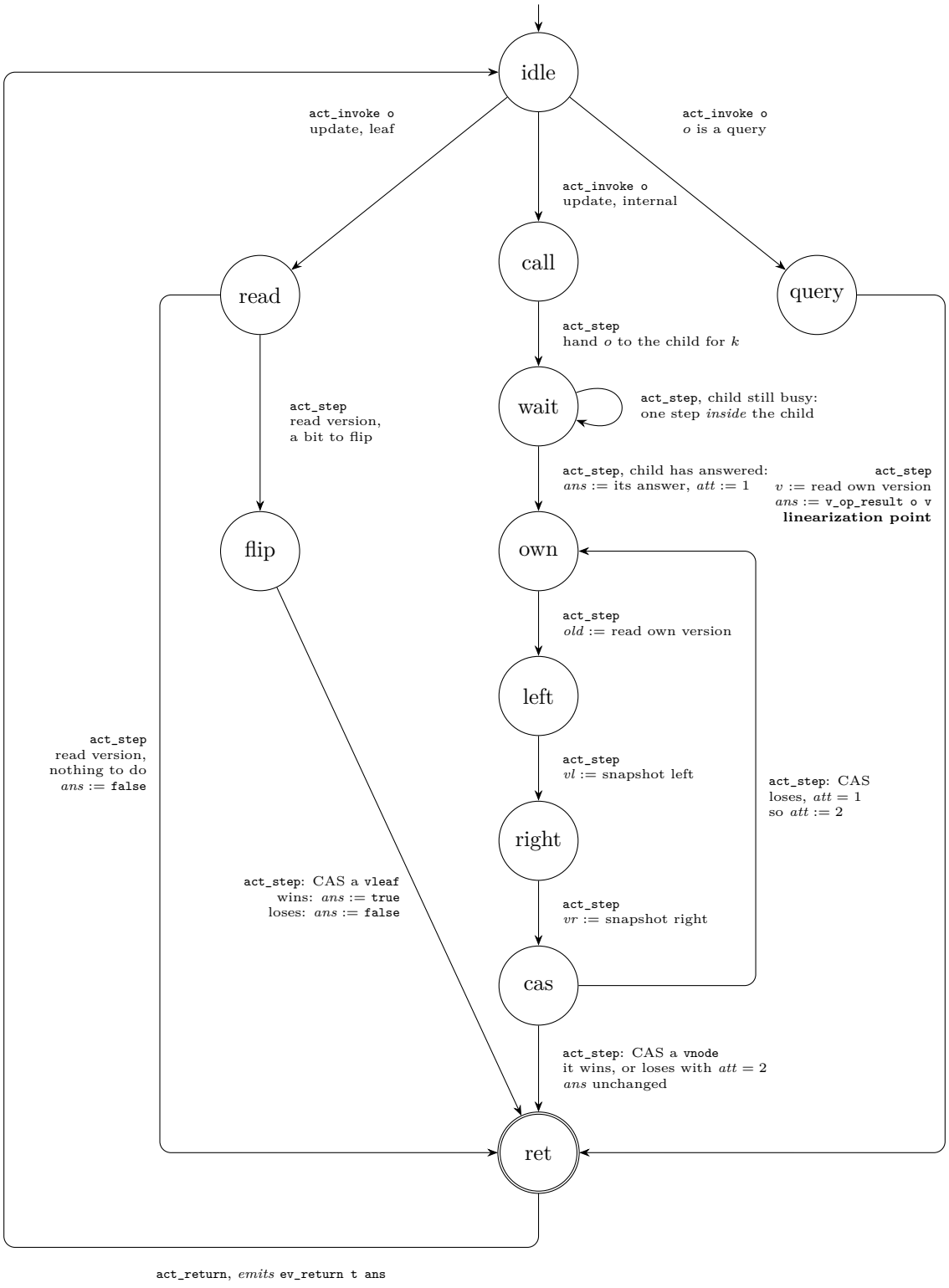


Key. Each circle is a value of `conc_progstate`, and the italic names are the local variables it carries: *o* the operation, *ans* the answer owed, *att* which of the two refresh attempts this is, and *old*, *vl*, *vr* the three versions a refresh reads. Every arrow is one atomic step, one read or one CAS; a local test costs no memory access and is decided inside the step that reaches it. The one exception is the loop on `wait`: each time round it, the thread spends its turn *inside* the child it handed *o* to. For an abstract child that is its single `act_step`, its linearization point; for a real child it is one arrow of this same diagram, one level down. The arrow `call` to `wait` is the child's invoke and the arrow `wait` to `own` is the child's return; neither emits an event, since both happen inside this node. All three arrows out of `idle` emit `ev_invoke t o` and are taken only when `op_ok i j o` holds. Every CAS compares tags rather than shape, and the two that win consume a tag from `fresh`. The four arrows `own`, `left`, `right`, `cas` are one `refresh`, and going round them twice is the double refresh. Any action not drawn is a stutter, leaving the configuration alone and emitting nothing: a disallowed `act_invoke`, an `act_return` anywhere but `ret`, and `act_step` at `idle` or at `ret`. Which of the three paths a thread takes is settled by the arrow it leaves `idle` on and cannot change afterwards, so no later state retests whether the node is a leaf; that is the lemma `conc_threads_ok_run`.